SWIFT SHIP TRACKER

User Story:

SwiftShip Tracker is a Salesforce-based parcel management solution that provides a centralised platform for customers to book parcels, track delivery status, and manage shipments through an easy-to-use interface and conversational AI support.

The system manages Parcel, Delivery, Sender, and Receiver information using Salesforce custom objects and their relationships. Salesforce Flows automate parcel-related operations and notifications. Agentforce AI and Prompt Builder allow customers to retrieve parcel information and interact with the system through conversational communication.

Delivery agents can update the parcel status in real time. Experience Cloud provides secure customer access, while security controls, reports, and dashboards help provide better visibility into parcel operations.

1. Introduction

1.1 Project Overview

SwiftShip Tracker is a Salesforce CRM solution developed to simplify parcel booking, shipment tracking, and delivery management. It supports the complete parcel lifecycle, starting from booking and dispatch to tracking and delivery confirmation. Customers can access their shipment information, while delivery agents can update parcel details and delivery status.

1.2 Purpose

The purpose of SwiftShip Tracker is to:

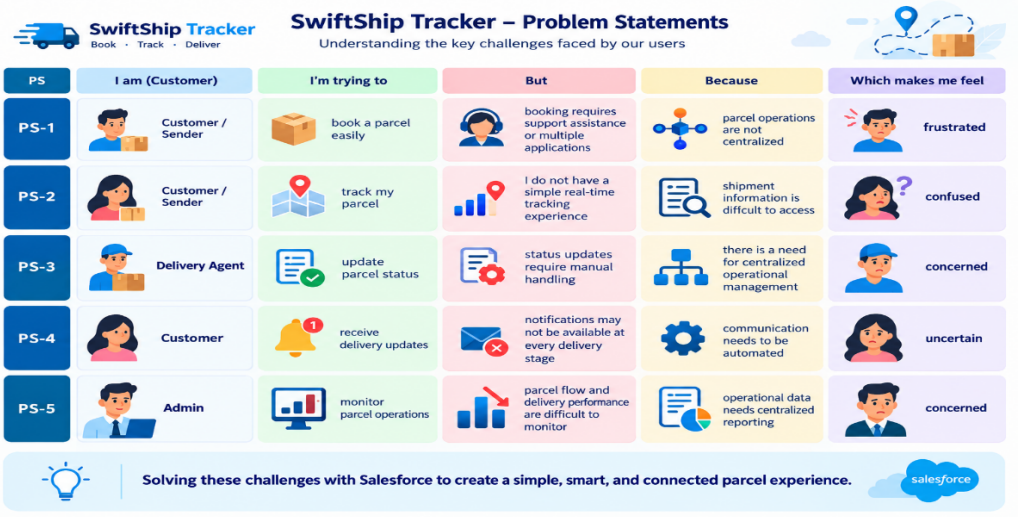
- Centralise parcel and delivery information.
- Simplify parcel booking and tracking.
- Provide real-time visibility of shipments.
- Automate delivery updates and customer notifications.
- Enable conversational parcel tracking using Agentforce AI and Prompt Builder.
- Improve agent efficiency through Salesforce Flows and dashboards.
- Provide secure, role-based access to customers, agents, support staff, and administrators.

2. IDEATION PHASE

2.1 Problem Statement

Customer Problem Statement Template

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Customer / Sender	Book a parcel easily	Booking requires support assistance or multiple applications	Parcel operations are not centralised	Frustrated
PS-2	Customer / Sender	Track my parcel	I do not have a simple real-time tracking experience	Shipment information is difficult to access	Confused
PS-3	Delivery Agent	Update parcel status	Status updates require manual handling	There is a need for centralised operational management	Concerned
PS-4	Customer	Receive delivery updates	Notifications may not be available at every delivery stage	Communication needs to be automated	Uncertain
PS-5	Admin	Monitor parcel operations	Parcel flow and delivery performance are difficult to monitor	Operational data needs centralised reporting	Concerned

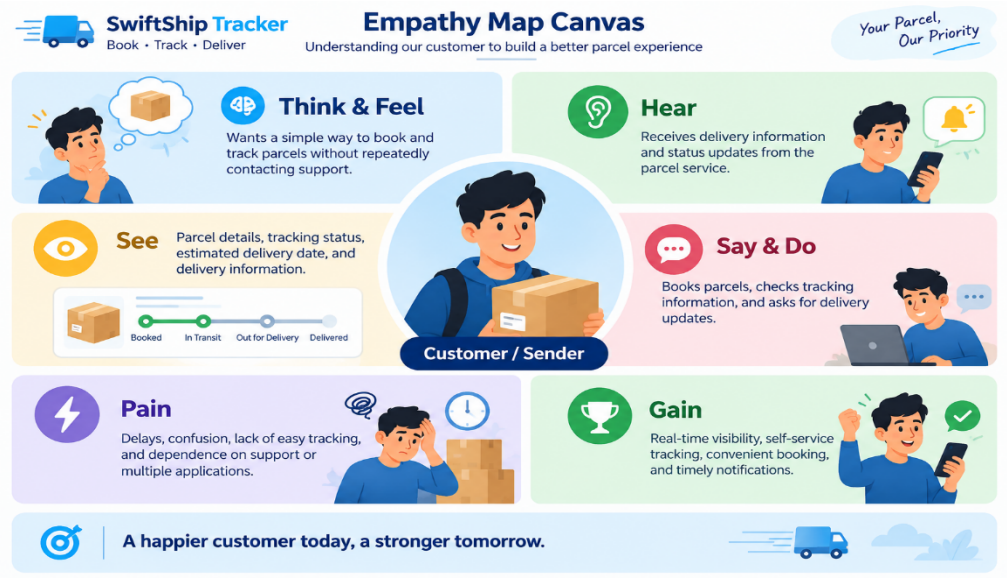


2.2 Empathy Map Canvas

User Persona: Customer / Sender

Empathy Map	Details
Think & Feel	Wants a simple way to book and track parcels without repeatedly contacting support.
Hear	Receives delivery information and status updates from the parcel

Empathy Map	Details
	service.
See	Parcel details, tracking status, estimated delivery date, and delivery information.
Say & Do	Books parcels, checks tracking information, and asks for delivery updates.
Pain	Delays, confusion, lack of easy tracking, and dependence on support or multiple applications.
Gain	Real-time visibility, self-service tracking, convenient booking, and timely notifications.



2.3 Brainstorming & Idea Prioritization

Ideas Considered

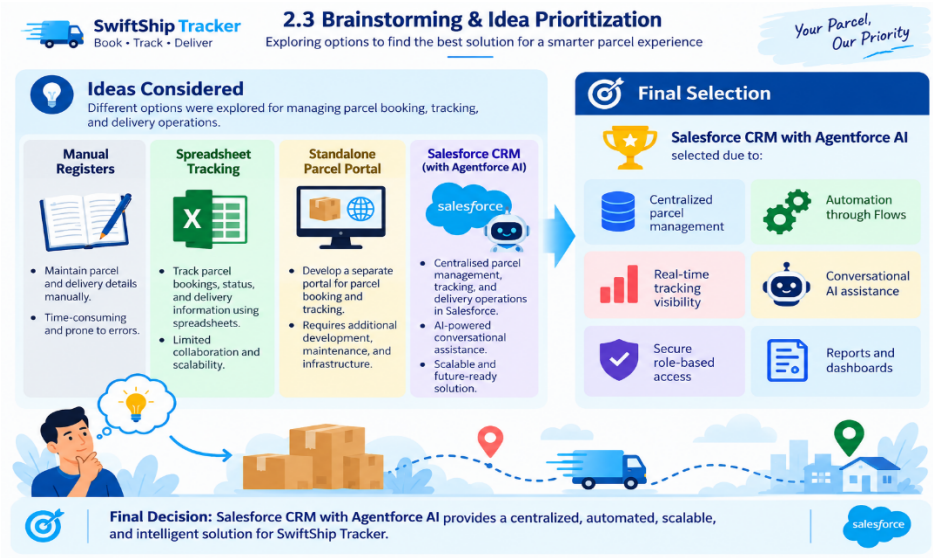
The following ideas were considered for managing parcel booking, tracking, and delivery operations:

- Manual registers
- Spreadsheet tracking
- Standalone parcel portal
- Salesforce CRM with Agentforce AI

Final Selection

Salesforce CRM was selected because it provides:

- Centralised parcel management
- Automation through Salesforce Flows
- Real-time tracking visibility
- Conversational AI assistance
- Secure role-based access
- Reports and dashboards



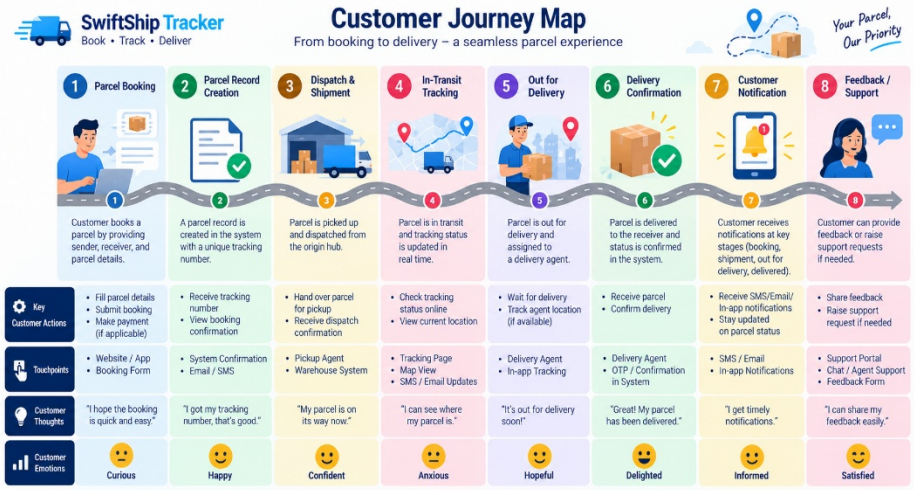
3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages

The customer journey for SwiftShip Tracker consists of the following stages:

1. Parcel Booking
2. Parcel Record Creation
3. Dispatch & Shipment
4. In-Transit Tracking
5. Out for Delivery
6. Delivery Confirmation
7. Customer Notification
8. Feedback / Support



3.2 Solution Requirements

Functional Requirements

The functional requirements of the SwiftShip Tracker system are listed below:

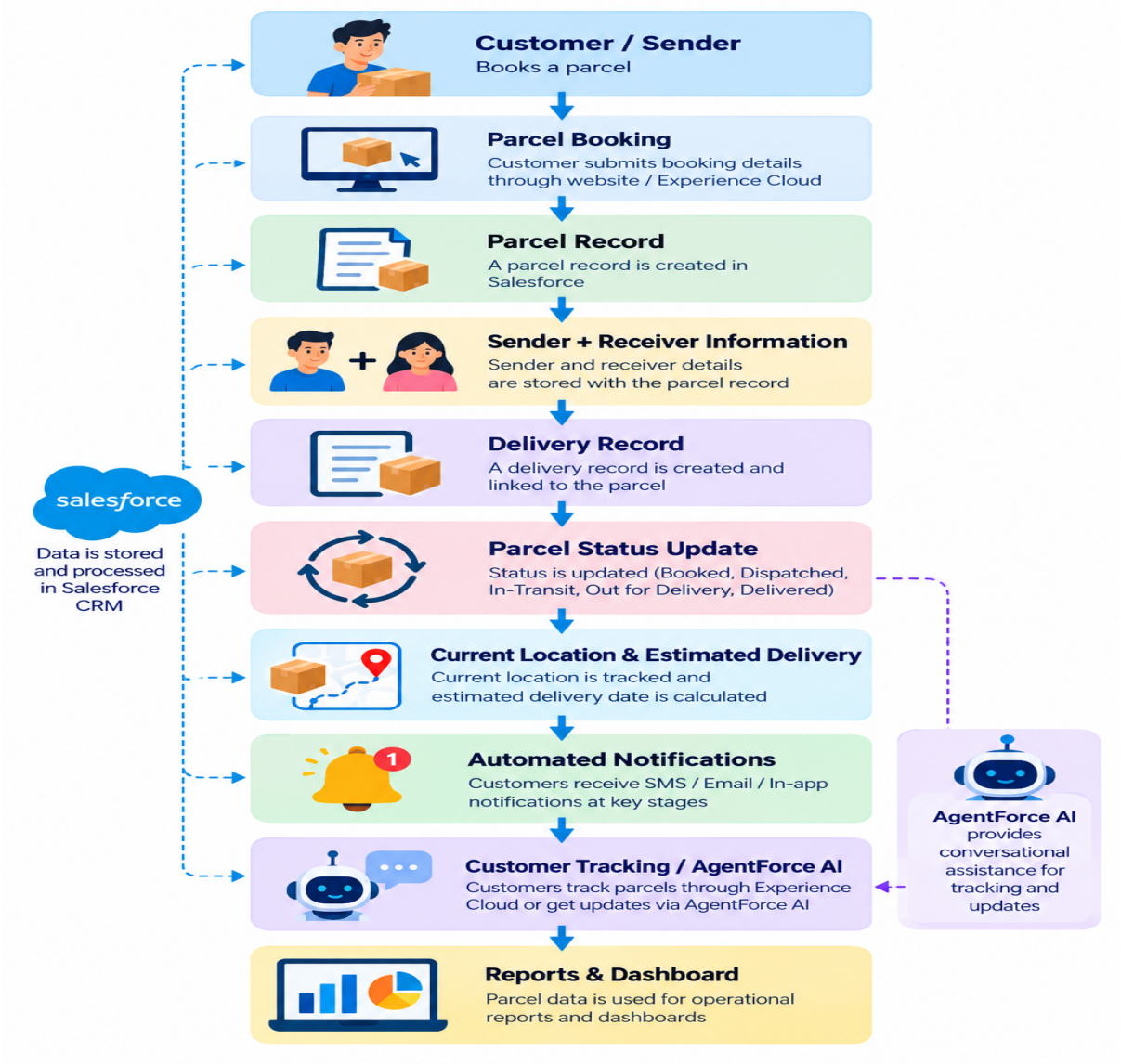
FR No.	Requirement
FR-1	Parcel booking and record creation
FR-2	Parcel status tracking
FR-3	Weight management
FR-4	Estimated delivery date management
FR-5	Sender information management
FR-6	Receiver information management
FR-7	Delivery and current-location tracking
FR-8	Agent parcel status updates
FR-9	Automated customer notifications
FR-10	AI-powered parcel tracking
FR-11	Reports and dashboards
FR-12	Customer access through Experience Cloud

Non-Functional Requirements

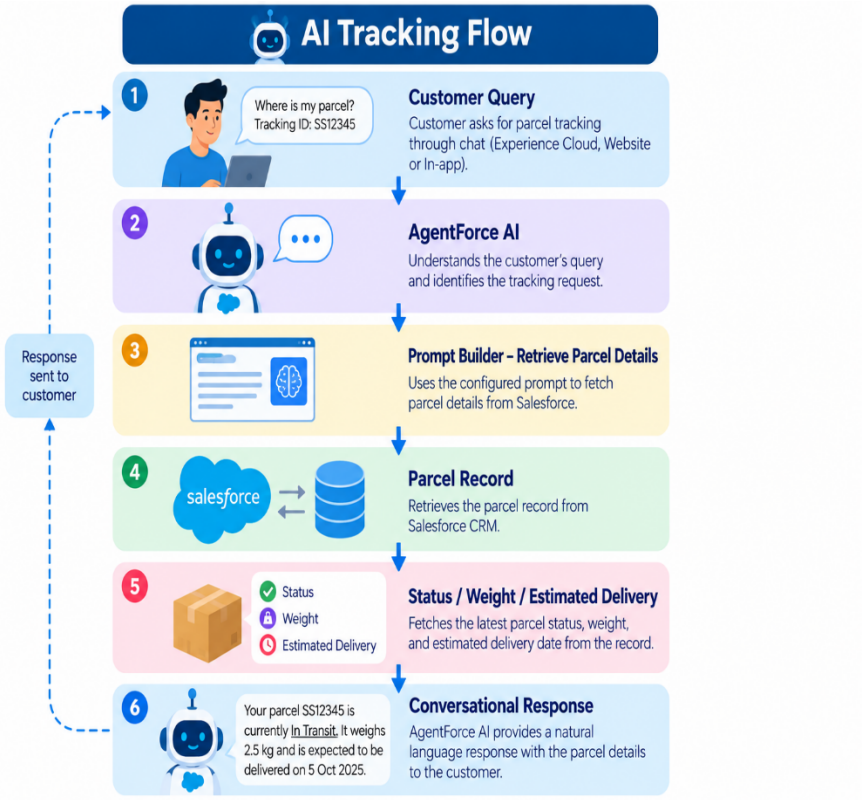
NFR	Description
Usability	Provides an intuitive Salesforce and customer-facing interface.
Security	Provides role-based and field-level access control.
Reliability	Provides accurate parcel status and delivery information.
Performance	Provides timely responses from Flows and Agentforce.
Availability	Provides cloud-based access through Salesforce.
Scalability	Supports the growth of parcel and delivery operations.
Data Integrity	Maintains controlled access and relationship-based parcel data.
Maintainability	Allows Flows, prompts, topics, and configurations to be updated.

3.3 Data Flow Diagram

Flow



AI Tracking Flow



The Prompt Builder implementation retrieves Parcel ID, Status, Weight, and Estimated Delivery Date information for tracking responses

3.4 Technology Stack

The SwiftShip Tracker project uses the following Salesforce technologies and components:

Layer	Technology / Platform
Salesforce Platform	Salesforce Developer Edition
UI	Lightning App Page, Lightning Record Pages, Experience Cloud
Database	Salesforce Custom & Standard Objects
Business Logic	Salesforce Flows, Apex Classes
Automation	Record-Triggered Flows, Auto-Launched Flows, Scheduled Flows
AI	Agentforce AI, Prompt Builder
Notifications	Email Alerts & Email Templates
Processing	Batch Apex

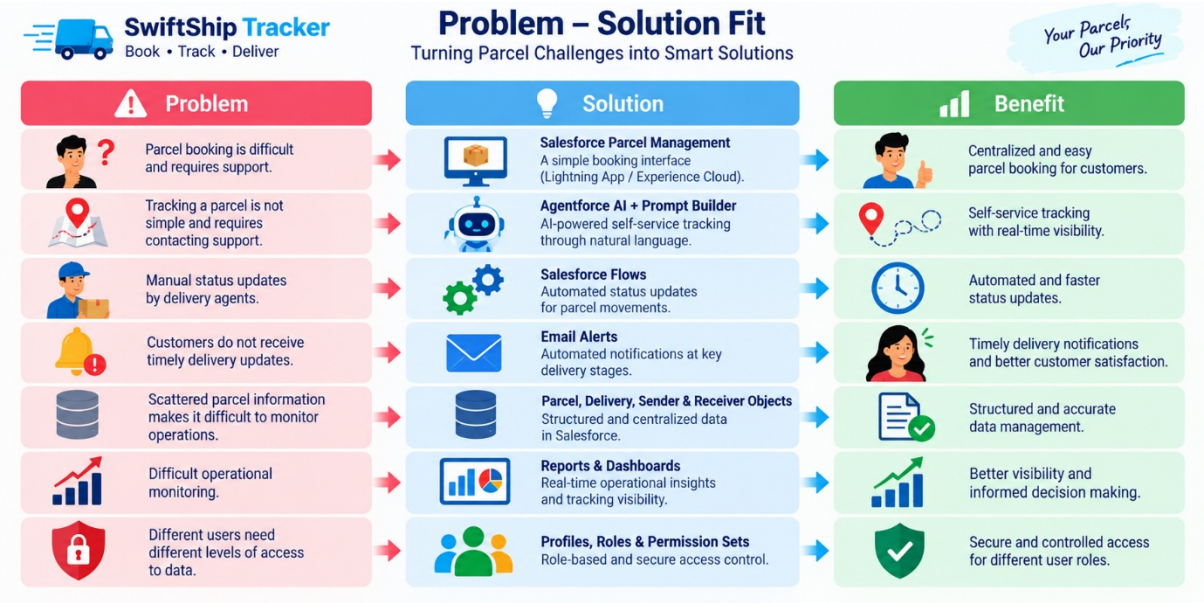
Layer	Technology / Platform
Security	Profiles, Permission Sets, Role Hierarchy, Field-Level Security, Sharing Rules
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The following table shows how the identified problems are addressed by the SwiftShip Tracker solution:

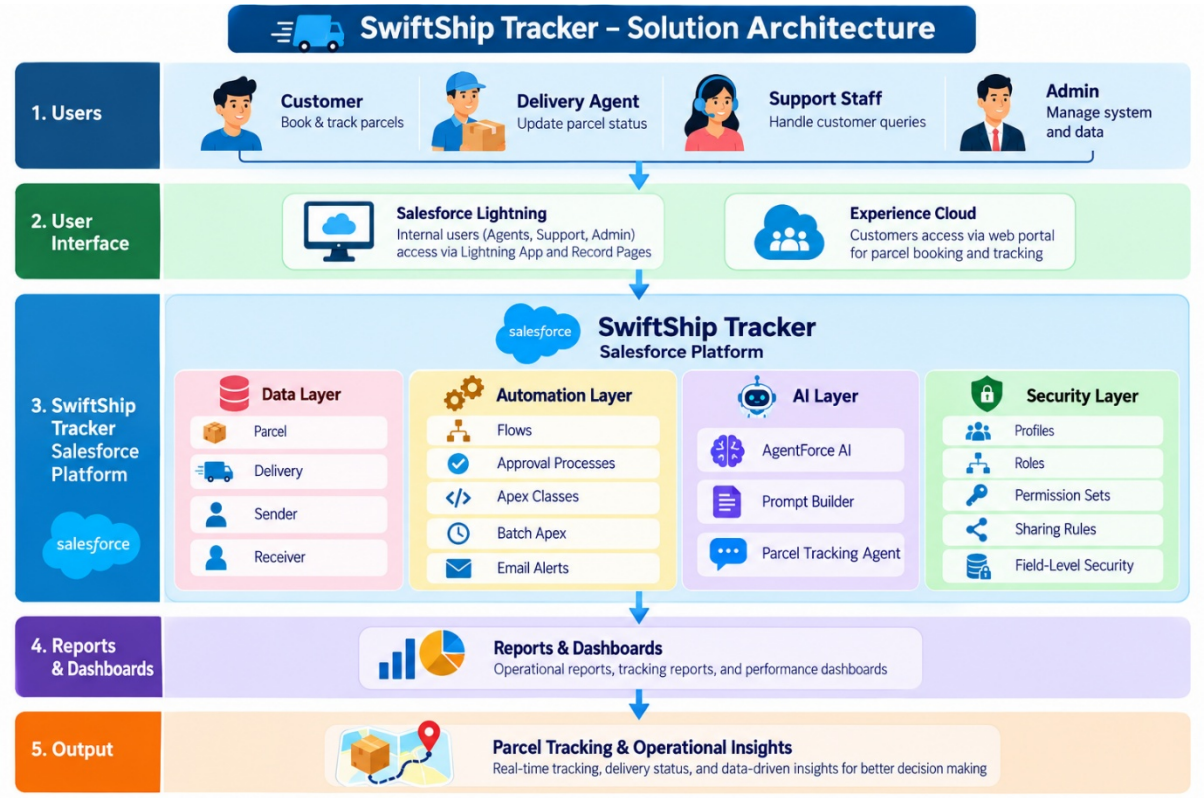
Problem	Solution	Benefit
Parcel booking is difficult	Salesforce Parcel Management	Centralised booking
Tracking requires support	Agentforce AI + Prompt Builder	Self-service tracking
Manual status updates	Salesforce Flows	Automated processes
Delivery communication gaps	Email Alerts	Timely notifications
Scattered shipment information	Parcel, Delivery, Sender & Receiver Objects	Structured data
Difficult operational monitoring	Reports & Dashboards	Better visibility
Different users need different access	Profiles, Roles & Permission Sets	Secure access



4.2 Proposed Solution

Parameter	Description
Functional Requirements	The system provides a centralised solution for parcel booking, tracking, delivery management, and related operations.
Problem	Parcel booking and tracking are difficult and inefficient.
Solution	A Salesforce-based Parcel Management System.
Innovation	Agentforce AI and Prompt Builder provide conversational self-service tracking, while Salesforce Flows automate parcel-related processes.
Impact	Provides centralised booking, real-time tracking visibility, timely notifications, and efficient delivery management.
Business Model	Supports parcel, customer, delivery-agent, support, and administrator operations.
Scalability	Supports growing parcel operations through Experience Cloud and integration with external logistics systems.

4.3 Solution Architecture



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project follows an **Agile methodology** for development and implementation.

- **Epics → Stories → Story Points:** The project requirements are organised into epics and user stories, with story points used to estimate the effort required.
- **Sprint-Based Execution:** Development is carried out in smaller sprints, with specific tasks and requirements completed in each sprint.
- **Velocity Calculation:** Sprint velocity is used to measure the amount of work completed and help estimate the work that can be completed in future sprints.

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Member
Sprint-1	Developer Setup	**USN-1**	As a system administrator, I want to create and configure the Salesforce environment so that I can build the SwiftShip Tracker parcel management system.	3	High	Abishek
Sprint-2	Data Modeling	**USN-2**	As an admin, I want to create Parcel, Delivery, Sender, and Receiver objects with proper relationships so that parcel and delivery data is stored and managed systematically.	5	High	Naveen
Sprint-2	Data Modeling	**USN-3**	As a staff member, I want dedicated tabs and a Lightning App for SwiftShip modules so that I can easily navigate and manage parcel and delivery operations.	5	High	Syed Faadil Ameen
Sprint-3	Automation	**USN-4**	As a system, I want to validate parcel details, mandatory fields, and booking inputs so that invalid or incomplete parcel records are prevented.	3	High	Trishan devjith
Sprint-3	Automation	**USN-5**	As a delivery coordinator, I want automated Flows to update parcel status and delivery information so that parcel processing requires minimal manual effort.	3	High	Trishan devjith
Sprint-4	Apex	**USN-6**	As a system, I want Apex triggers and classes to automatically perform critical parcel and delivery actions so that important processes are	5	High	Viswanathan

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Member
			handled accurately.			
Sprint-4	Apex	**USN-7**	As an inventory/operations manager, I want Batch Apex jobs to process and update parcel or delivery records automatically so that operational data remains accurate and up to date.	5	High	Abishek
Sprint-5	Reports	**USN-8**	As a manager, I want reports showing parcel status, delivery performance, and operational metrics so that I can monitor SwiftShip operations and make informed decisions.	4	High	Abishek
Sprint-6	Dashboards & AI	**USN-9**	As a manager, I want dashboards showing parcel status, delivery performance, and key metrics, while Agentforce provides AI-powered tracking assistance, so that I can monitor operations efficiently.	4	Medium	Syed faadil Ameen

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	3	Half Day	29 Sep 2026	29 Sep 2026	3	29 Sep 2026
Sprint-2	10	Half Day	29 Sep 2026	29 Sep 2026	10	29 Sep 2026
Sprint-3	6	Half Day	30 Sep 2026	30 Sep 2026	6	30 Sep 2026

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-4	10	Half Day	30 Sep 2026	30 Sep 2026	10	30 Sep 2026
Sprint-5	4	Half Day	01 Oct 2026	01 Oct 2026	4	01 Oct 2026
Sprint-6	4	Half Day	01 Oct 2026	01 Oct 2026	4	01 Oct 2026

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis & Planning

Objective

Customers often face delays and confusion while booking, tracking, and managing parcel deliveries because they need to contact support or use multiple applications. SwiftShip Tracker addresses this by providing a conversational AI agent that allows customers to book parcels, get live tracking information, and manage deliveries in one place.

Key Requirements

- Gather requirements from end users and delivery agents.
- Define the system scope and identify the main user personas: **Customer, Agent, and Admin**.
- Design the data model with relationships between **Parcel, Sender, Receiver, and Delivery**.
- Identify key use cases such as **parcel booking, parcel tracking, and automated delivery updates**.
- Select the required Salesforce tools, including **Salesforce Flows, Prompt Builder, and Agentforce AI**.

Defining Project Scope & Objectives

Project Scope

The scope of the SwiftShip Tracker project includes:

1. Build a scalable CRM solution for parcel delivery businesses to manage customer bookings, shipments, and delivery operations efficiently.
2. Support the complete parcel lifecycle, from **parcel booking → dispatch → tracking → delivery confirmation**.
3. Include a comprehensive data model, automation using **Flows, Apex, and Email Alerts**, AI-powered **Prompt Builder**, and reporting dashboards for effective management.
4. Provide role-based access for **admins, delivery agents, and customers** using Salesforce Profiles, Permission Sets, and Experience Cloud.
5. Provide real-time parcel visibility for customers and simplify operational tracking for agents through **AI-integrated conversational support**.

Objectives Summary

- Streamline parcel operations by automating booking, tracking, and delivery notifications to reduce manual updates.
- Improve customer satisfaction by providing transparent, real-time shipment status and self-service tracking options.
- Improve agent efficiency through centralised dashboards and guided Flows for quick status updates.
- Provide business insights through reports and dashboards showing parcel trends, delivery performance, and agent activity.
- Ensure security and reliability by implementing Salesforce role hierarchies, sharing rules, and audit-based data access.

Gathering & Analysing User Needs

Users Involved

- **Customers / Senders** – End users who book parcels, track deliveries, and receive real-time updates through the portal or AI agent.
- **Delivery Agents** – Manage assigned parcels, update delivery status, and ensure timely handover to customers.
- **Customer Support Team** – Handle delivery-related issues, respond to customer queries, and ensure smooth communication.
- **Admin Team** – Configure the Salesforce environment, manage user roles and security settings, and monitor operational performance.
- **System Integrator / Developer** – Responsible for implementing automation, setting up the AI agent, and managing backend integrations.

Key Functional Needs

- Provide a seamless parcel booking and tracking interface for customers through the Salesforce Experience Site or AI agent assistance.
- Allow delivery agents to update parcel status in real time, from **Booked → In Transit → Out for Delivery → Delivered**.
- Send automated notifications and email alerts to customers whenever the delivery stage is updated.
- Manage Sender and Receiver details, including address, contact, and email information for communication.
- Integrate an AI Agent through Prompt Builder to handle real-time parcel status queries and provide automated assistance.
- Provide reports and dashboards to help admins monitor parcel flow, delivery performance, and agent activity.
- Implement security controls using role-based access, Profiles, and Permission Sets for different user levels.

Tools Used

- **Salesforce Developer Org** – Used for configuration, development, and testing.
- **Prompt Builder (Agentforce AI)** – Used to create intelligent parcel tracking prompts and responses.
- **Salesforce Flow Builder** – Used to automate booking, tracking, and delivery updates.
- **Google Sheets / Docs** – Used for initial data planning, requirement documentation, and system mapping.

Identifying Key Salesforce Features & Tools Required

Salesforce Features Planned

The following Salesforce features and tools were identified and used for the development of the SwiftShip Tracker system:

- **Custom Objects:** Parcel__c, Delivery__c, Sender__c, Receiver__c
- **Standard Objects:** Account, Contact, Case, Task, EmailMessage, User
- **Automations:** Record-Triggered Flows, Auto-Launched Flows, Approval Processes, and Scheduled Flows
- **Apex:** Apex Classes are used for automated parcel operations, such as Lead-to-Parcel mapping and delivery notifications. Batch Apex is used for periodic updates or notifications.

- **User Interface:** Lightning App Page for the SwiftShip Tracker App, Dynamic Forms for Parcel Details, Lightning Record Pages, and Experience Cloud for customer access.
- **Agentforce AI & Prompt Builder:** An integrated AI agent is used to fetch parcel information based on the Parcel ID and respond to customer queries related to tracking and delivery status.
- **Email Services:** Custom Email Alerts and Email Templates are used for booking confirmations, delivery updates, and completion acknowledgements.
- **Security:** Profiles, Permission Sets, Role Hierarchies, Field-Level Security, and Sharing Rules are used to provide controlled access for Admin, Agent, and Customer roles.

Designing Data Model and Security Model

Data Model

The SwiftShip Tracker system uses Salesforce custom objects to store and manage parcel, delivery, sender, and receiver information. The main objects in the data model are:

1. Parcel__c – Custom Object

- Stores core parcel details such as **Parcel ID, Status, Weight, and Estimated Delivery Date**.
- Linked to **Sender__c** and **Receiver__c** objects to fetch sender and receiver details.
- Connected with the **Delivery__c** object to track the parcel's current location and status.

2. Delivery__c – Custom Object

- Stores delivery-specific information such as **Current Location, Estimated Delivery Date, and Linked Parcel**.
- Connected with **Sender__c** and **Parcel__c** for relationship-based tracking.
- Used by delivery agents to update and monitor the delivery status during the parcel lifecycle.

3. Sender__c – Custom Object

- Captures sender-related information such as **Address, Contact Number, and Email**.

- Linked to multiple parcels to track sender history and associated deliveries.
- Used for sending booking confirmations and delivery updates through automated emails.

4. Receiver__c – Custom Object

- Stores receiver details such as **Address, Contact, and Email** for communication and delivery confirmation.
- Linked to **Sender__c** and **Parcel__c** to associate both ends of the shipment process.
- Used by delivery agents for delivery reference and for sending automated email notifications after delivery completion.

Security Model Design

The SwiftShip Tracker system uses Salesforce security features to control access to data based on the user's role and responsibilities.

1. Role Hierarchy

The role hierarchy is structured as:

Administrator → Delivery Manager → Agent → Customer

2. Profiles

- **Admin Profile:** Provides full access to all objects and records.
- **Delivery Agent Profile:** Provides Read/Edit access to Parcel, Delivery, Sender, and Receiver objects.
- **Customer Profile:** Provides read-only access to the customer's own parcel details through Experience Cloud.
- **Support Staff Profile:** Provides access to Cases and Parcel details for resolving customer issues.

3. Permission Sets

Permission Sets are used to provide additional privileges when required. Examples include:

- **Update Delivery Status**
- **Access Agent Console**

4. Record-Level Security

Sharing Rules are applied to **Parcel__c** and **Delivery__c** to provide controlled access to agents and managers.

Customers can view only the records linked to their own Sender or Receiver details.

5. Field-Level Security

Sensitive fields such as **Sender Email, Receiver Contact, and Location Coordinates** are restricted to Admin and Agent profiles to protect sensitive information.

Phase 2: Salesforce Development – Backend & Configurations

This phase focused on developing the core functionality of the SwiftShip Tracker system. Custom objects were created for **Parcel, Sender, Receiver, and Delivery**, along with the required relationships between them. Automation was implemented using **Salesforce Flows** for parcel status updates and **Approval Processes**.

Milestone 1 – Salesforce Account Introduction

Salesforce is a customer success platform that provides tools to help organisations manage their customer-related activities. It supports different business functions such as **sales, customer service, marketing, analysis, and customer engagement**.

As part of this milestone, the Salesforce environment was introduced and configured as the foundation for developing the SwiftShip Tracker application.

Creating Developer Account

A Salesforce Developer Org was created to develop and configure the SwiftShip Tracker application.

Steps to Create a Developer Org

1. Go to the Salesforce Developer signup page: Salesforce Developer Signup
2. On the signup form, enter the following details:
    - **First Name & Last Name:** Abishek S
    - **Email:** poke936o@gmail.com
    - **Job Title:** Developer
    - **Company:** Sree Sastha Institute Of Engineering And Technology
    - **Country:** India
3. After entering all the required details, click **Sign Me Up**.

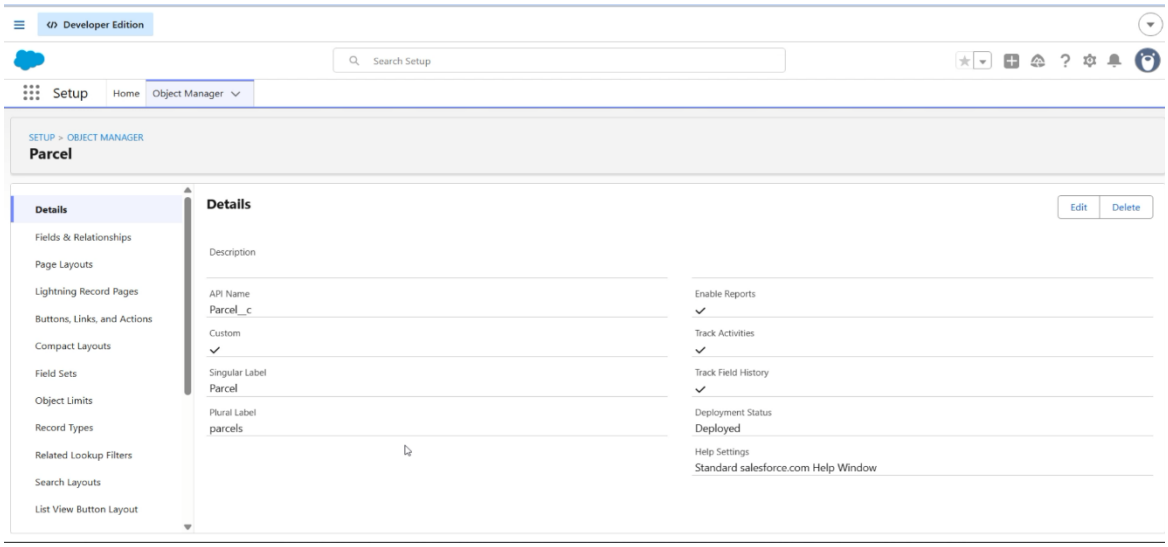
4. Open the inbox of the email address used during registration.
5. Open the Salesforce verification email and click **Verify Account** to activate the account.
    - The verification email may take **10–30 minutes** to arrive and, in some cases, may take up to **2 hours**.
6. Click **Verify Account** in the email.
7. Create a password, select and answer the required security question, and click **Change Password**.
8. After completing the password setup, you will be redirected to the **Salesforce Setup** page.

Milestone 2 – Objects Creation

Custom objects were created in Salesforce to store and manage the required parcel information for the SwiftShip Tracker system.

Steps to Create a Custom Object

1. From the **Salesforce Setup** page, open **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter the **Label** as Parcel.
4. Enter the **Plural Label** as Parcels.
5. Enter the **Record Name** as Parcel Name.
6. Select the **Data Type** as **Text** for the Parcel Name.
7. Select **Allow Reports**.
8. Select **Allow Search**.
9. Select **Allow Track Field History**.
10. Click **Save and New** to save the Parcel object and continue creating the next object.

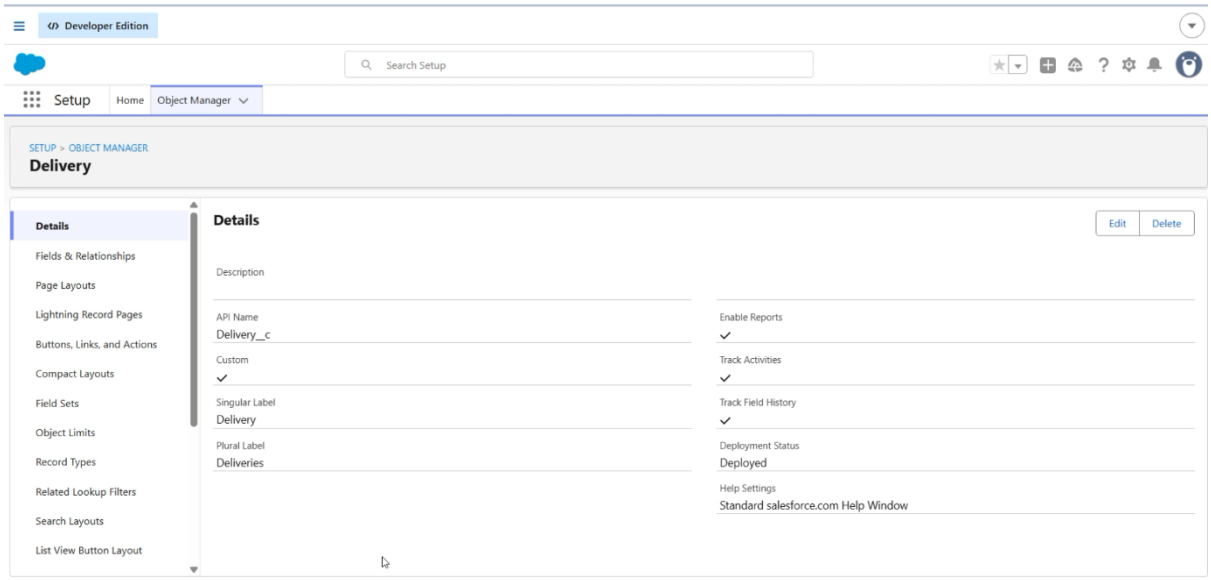


Creating Delivery Object

A **Delivery** custom object was created in Salesforce to store and manage delivery-related information for the SwiftShip Tracker system.

Steps to Create the Delivery Object

1. From the **Salesforce Setup** page, open **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter the **Label** as Delivery.
4. Enter the **Plural Label** as Deliveries.
5. Enter the **Record Name** as DeliveryName.
6. Select the **Data Type** as **Text** for DeliveryName.
7. Select **Allow Reports**.
8. Select **Allow Search**.
9. Select **Allow Track Field History**.
10. Click **Save and New** to save the Delivery object and continue creating the next object.

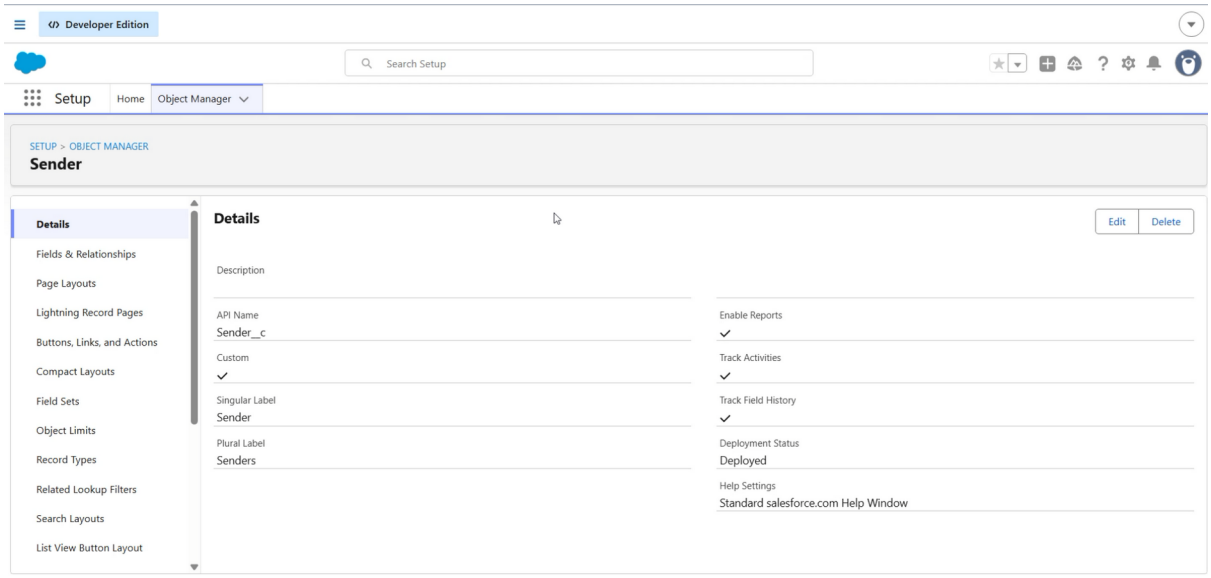


Creating Sender Object

A **Sender** custom object was created in Salesforce to store and manage sender-related information for the SwiftShip Tracker system.

Steps to Create the Sender Object

1. From the **Salesforce Setup** page, open **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter the **Label** as Sender.
4. Enter the **Plural Label** as Sender's.
5. Enter the **Record Name** as SenderName.
6. Select the **Data Type** as **Text** for SenderName.
7. Select **Allow Reports**.
8. Select **Allow Search**.
9. Select **Allow Track Field History**.
10. Click **Save and New** to save the Sender object and continue creating the next object.

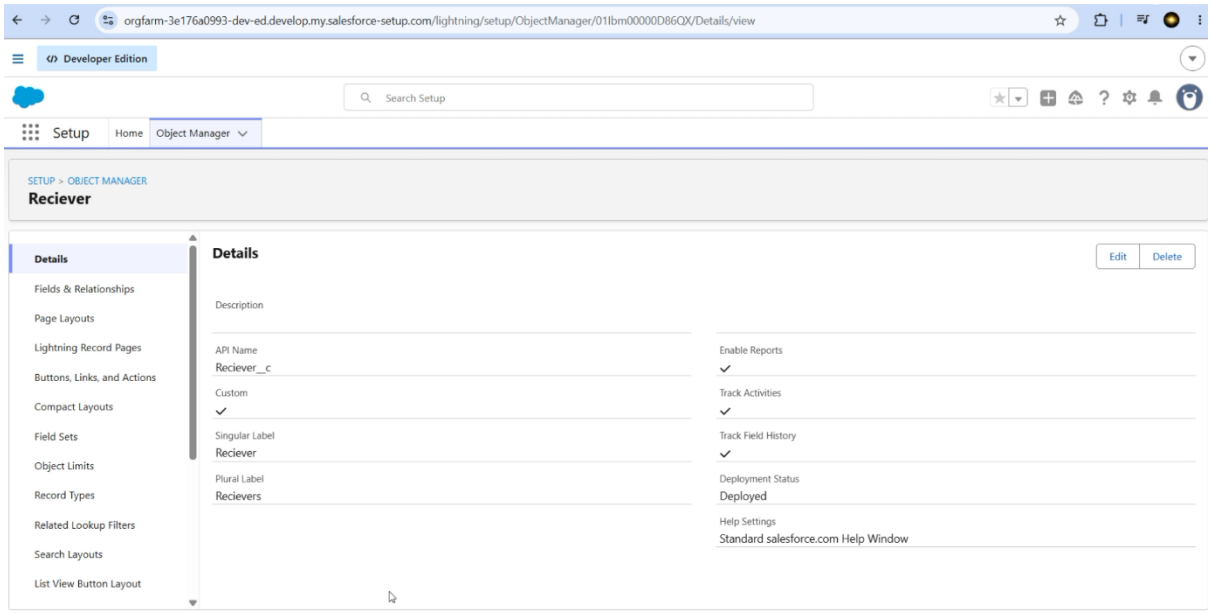


Creating Receiver Object

A **Receiver** custom object was created in Salesforce to store and manage receiver-related information for the SwiftShip Tracker system.

Steps to Create the Receiver Object

1. From the **Salesforce Setup** page, open **Object Manager**.
2. Click **Create** and select **Custom Object**.
3. Enter the **Label** as Receiver.
4. Enter the **Plural Label** as Receiver's.
5. Enter the **Record Name** as ReceiverName.
6. Select the **Data Type** as **Text** for ReceiverName.
7. Select **Allow Reports**.
8. Select **Allow Search**.
9. Select **Allow Track Field History**.
10. Click **Save and New** to save the Receiver object and continue creating the next object.



Milestone 3 – Tabs

Creating a Tab for the Parcel Object

A custom tab was created for the **Parcel** object to make it easily accessible within Salesforce.

Steps to Create the Parcel Tab

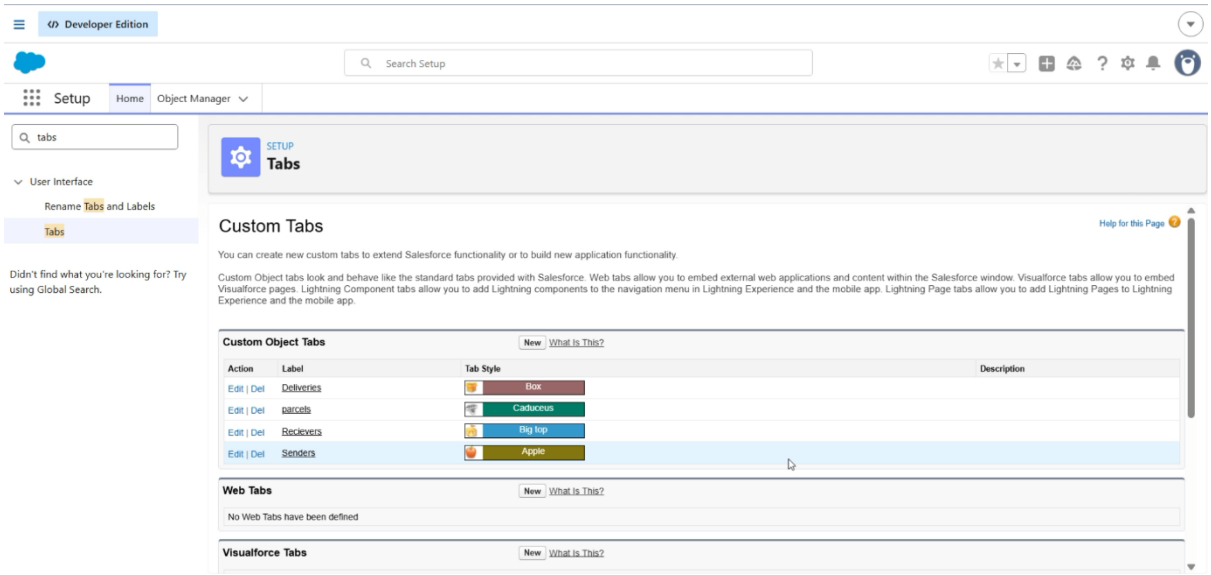
1. Go to the **Salesforce Setup** page.
2. Type **Tabs** in the **Quick Find** search bar and open **Tabs**.
3. Under **Custom Object Tabs**, click **New**.
4. Select **Parcel** as the object and choose the required **Tab Style**.
5. Click **Next**.
6. On the **Add to Profiles** page, keep the default settings and click **Next**.
7. On the **Add to Custom Apps** page, uncheck **Include Tab**.
8. Make sure **Append tab to the user's existing personal customizations** is selected.
9. Click **Save** to create the Parcel tab.

More Tab Creation

Custom tabs were created for the **Delivery, Sender, and Receiver** objects to make them easily accessible in Salesforce.

Steps to Create Additional Tabs

1. Go to **Setup** and type **Tabs** in the **Quick Find** box.
2. Click **Tabs** under the **User Interface** section.
3. Under **Custom Object Tabs**, click **New**.
4. Select the required object (**Delivery, Sender, or Receiver**) and choose a **Tab Style**.
5. Click **Next**.
6. Keep the **Add to Profiles** settings as default and click **Next**.
7. On the **Add to Custom Apps** page, uncheck **Include Tab**.
8. Ensure **Append tab to the user's existing personal customizations** is checked.
9. Click **Save**.
10. Repeat the same steps for each of the **Delivery, Sender, and Receiver** objects.



Milestone 4 – Fields & Relationships

Fields and relationships were created for the custom objects created in the previous milestone. These fields store the required parcel information and establish relationships between the Parcel and other objects.

Creating Fields and Relationships for the Parcel Object

1. Go to **Setup** in Salesforce.
2. In **Object Manager**, search for and select the **Parcel** object.
3. Click **Fields & Relationships**.
4. Click **New** to create a new field.

Field 1: Parcel ID

Enter the following details:

- **Data Type:** Auto Number
- **Display Format:** P-{000}

5. Click **Next**.
6. Click **Next** again to proceed with the field settings.
7. Click **Save & New**.

The Auto Number field automatically generates a unique Parcel ID for each Parcel record, such as P-001, P-002, and P-003.

Creating the Remaining Parcel Fields

Follow the same process to create the remaining fields of the **Parcel** object:

Field 2: Status

1. From **Fields & Relationships**, click **New**.
2. Select the required data type for the **Status** field.
3. Enter the field label as **Status**.
4. Configure the required field options.
5. Click **Next**.
6. Click **Next** again.
7. Click **Save & New**.

Field 3: Weight

1. Click **New** under **Fields & Relationships**.
2. Select the required data type for the **Weight** field.
3. Enter the field label as **Weight**.
4. Configure the required field options.
5. Click **Next**.

6. Click **Next** again.
7. Click **Save & New**.

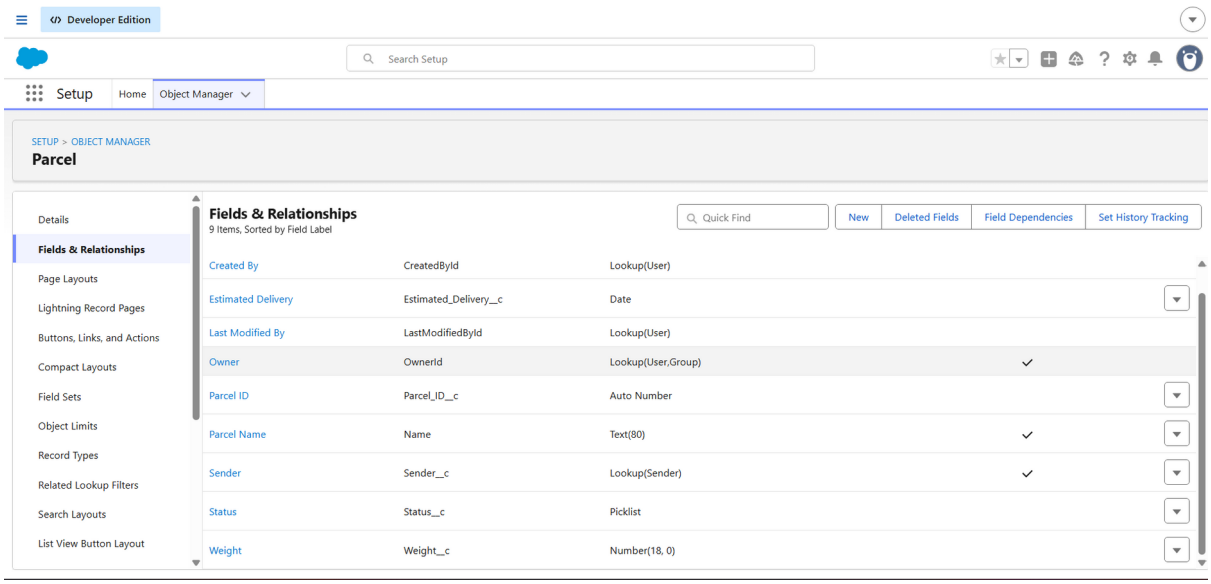
Field 4: Estimated Delivery

1. Click **New** under **Fields & Relationships**.
2. Select the required data type for the **Estimated Delivery** field.
3. Enter the field label as **Estimated Delivery**.
4. Configure the required field options.
5. Click **Next**.
6. Click **Next** again.
7. Click **Save & New**.

Field 5: Sender

1. Click **New** under **Fields & Relationships**.
2. Select the required relationship data type for the **Sender** field.
3. Enter the field label as **Sender**.
4. Select the **Sender** object as the related object.
5. Configure the required relationship settings.
6. Click **Next**.
7. Click **Next** again.
8. Click **Save**.

The **Sender** field establishes a relationship between the **Parcel** and **Sender** objects, allowing the sender information to be associated with the corresponding parcel record.



Creating Fields and Relationships for the Delivery Object

1. Go to **Setup** in Salesforce and open **Object Manager**.
2. Search for and select the **Delivery** object.

Field 1: Current Location

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Geolocation.
- Click **Next**.
- Enter the **Field Label** as **Current Location**.
- Click **Next** → **Next** → **Save & New**.

Field 2: Estimated Delivery

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Date.
- Click **Next**.
- Enter the **Field Label** as **Estimated Delivery**.
- Click **Next** → **Next** → **Save & New**.

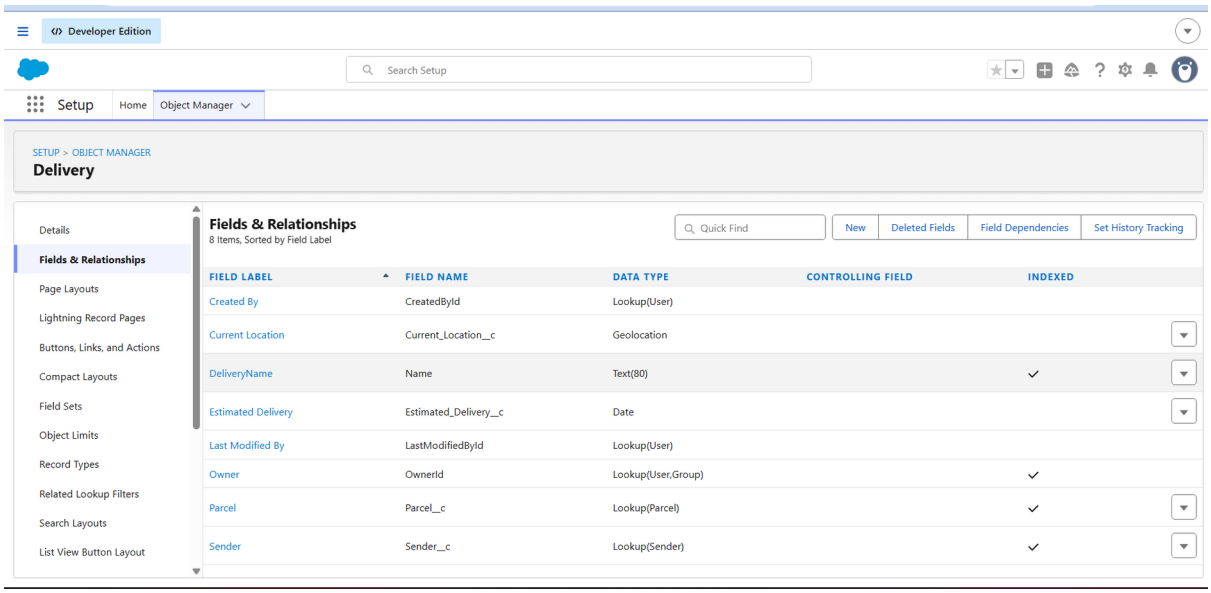
Field 3: Sender

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Lookup Relationship.

- Click **Next**.
- Select **Sender** as the related object.
- Enter the **Field Label** as **Sender**.
- Click **Next → Next → Save & New**.

Field 4: Parcel

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Lookup Relationship.
- Click **Next**.
- Select **Parcel** as the related object.
- Enter the **Field Label** as **Parcel**.
- Click **Next → Next → Save**.



Creating Fields and Relationships for the Sender Object

1. Go to **Setup** in Salesforce and open **Object Manager**.
2. Search for and select the **Sender** object.

Field 1: Sender Address

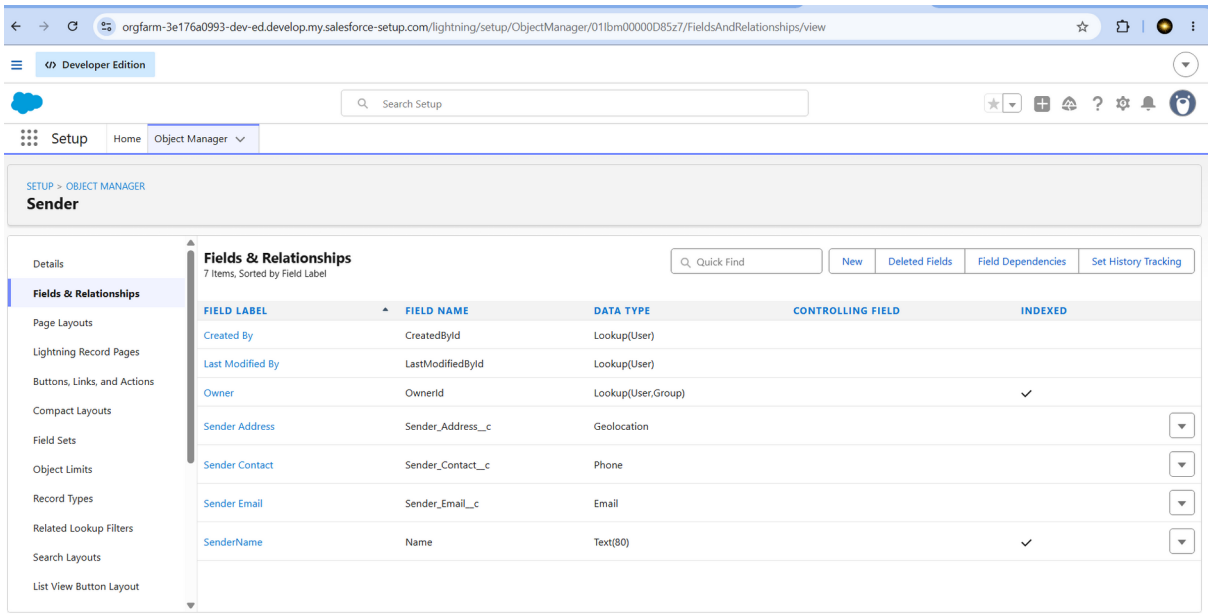
- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Geolocation.
- Click **Next**.
- Enter the **Field Label** as **Sender Address**.
- Click **Next → Next → Save & New**.

Field 2: Sender Contact

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Phone.
- Click **Next**.
- Enter the **Field Label** as **Sender Contact**.
- Click **Next** → **Next** → **Save & New**.

Field 3: Sender Email

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Email.
- Click **Next**.
- Enter the **Field Label** as **Sender Email**.
- Click **Next** → **Next** → **Save**.



Creating Fields and Relationships for the Receiver Object

1. Go to **Setup** in Salesforce and open **Object Manager**.
2. Search for and select the **Receiver** object.

Field 1: Receiver Address

- Go to **Fields & Relationships** → click **New**.

- Select **Data Type:** Geolocation.
- Click **Next**.
- Enter the **Field Label** as **Receiver Address**.
- Click **Next → Next → Save & New**.

Field 2: Receiver Contact

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Phone.
- Click **Next**.
- Enter the **Field Label** as **Receiver Contact**.
- Click **Next → Next → Save & New**.

Field 3: Receiver Email

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Email.
- Click **Next**.
- Enter the **Field Label** as **Receiver Email**.
- Click **Next → Next → Save & New**.

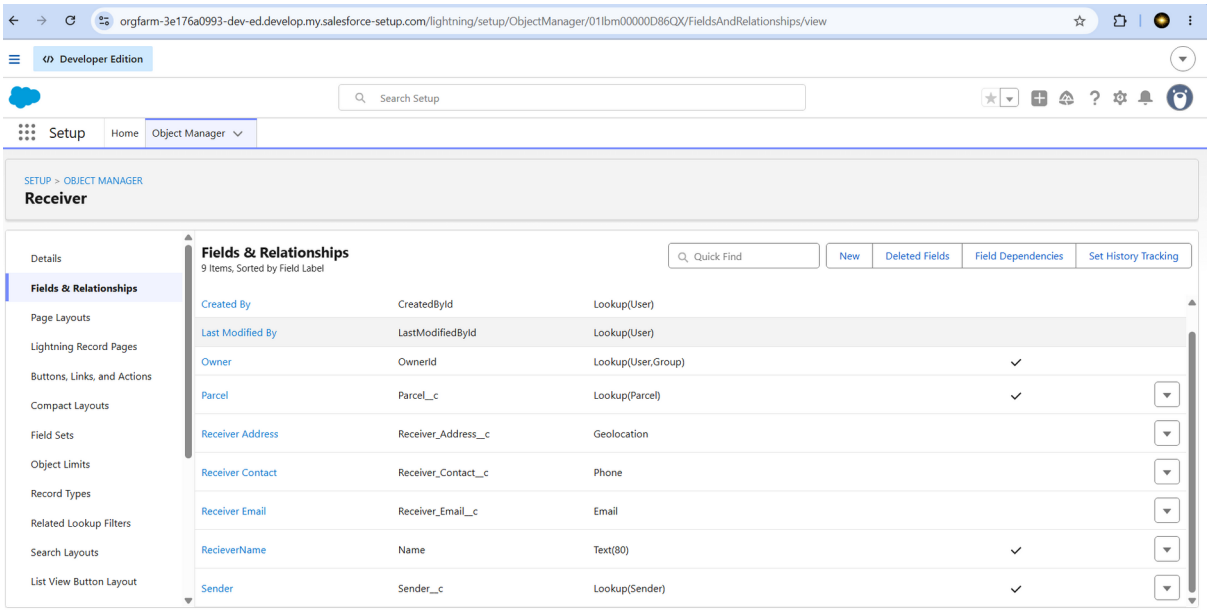
Field 4: Sender

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Lookup Relationship.
- Click **Next**.
- Select **Sender** as the related object.
- Enter the **Field Label** as **Sender**.
- Click **Next → Next → Save & New**.

Field 5: Parcel

- Go to **Fields & Relationships** → click **New**.
- Select **Data Type:** Lookup Relationship.
- Click **Next**.
- Select **Parcel** as the related object.
- Enter the **Field Label** as **Parcel**.

- Click **Next → Next → Save**.



Milestone 5 – Prompt Builder

Designing Prompts for Agent Building

Prompt Builder was used to create a prompt template for retrieving parcel details based on the user's input.

Creating the Prompt Template

1. Go to **Setup** in Salesforce.
2. In the **Quick Find** box, search for **Prompt Templates**. Alternatively, in Agent Console, go to **Agent Builder → Prompt Templates**.
3. Click **New Prompt Template**.
4. Select **Record Summary** as the template type.

Enter Prompt Template Details

Enter the following details:

- **Label:** Retrieve Parcel Details
- **API Name:** Retrieve_Parcel_Details
- **Description:** Retrieves parcel details where Parcel_ID matches the user input.

Prompt Template Content

In the **Prompt Template Text** area, enter the following content:

Retrieve details from Parcel c where Parcel_ID__c matches user input. Return response in this format:

Parcel Tracking Update

📦 Parcel Tracking Update

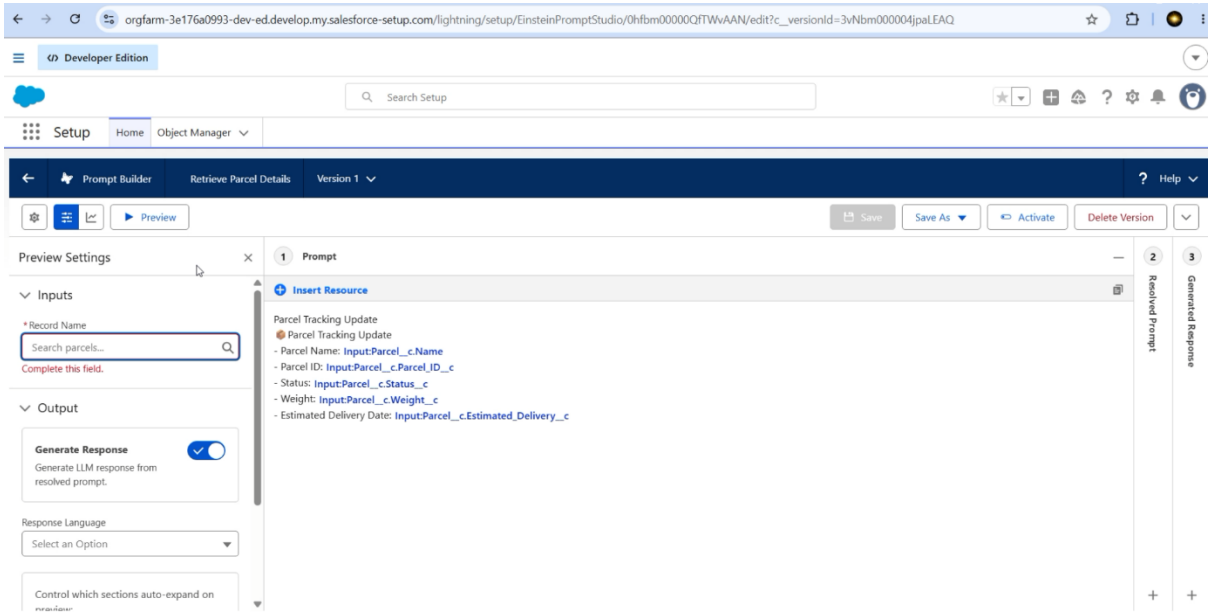
- Parcel Name:{!$Input:Parcel__c.Name}

- Parcel ID:{!$Input:Parcel__c.Parcel_ID__c}

- Status:{!$Input:Parcel__c.Status__c}

- Weight:{!$Input:Parcel__c.Weight__c}

- Estimated Delivery Date:{!$Input:Parcel__c.Estimated_Delivery_Date__c}



Adding Resource Mapping

1. Scroll down to the **Resources / Insert Resources** section.
2. Click **Insert Resource** and select **Object**.
3. Select **Parcel__c**.
4. Add the following fields:
    - **Name**
    - **Parcel_ID__c**
    - **Status__c**

- **Weight__c**
- **Estimated_Delivery_Date__c**

5. Click **Insert Resource** or **Add** for each field.
6. Click **Save**.

Activating the Prompt Template

After saving the prompt template:

1. Click **Activate**.
2. The Prompt Template is now available for use with the **Agentforce agent or Flow**.

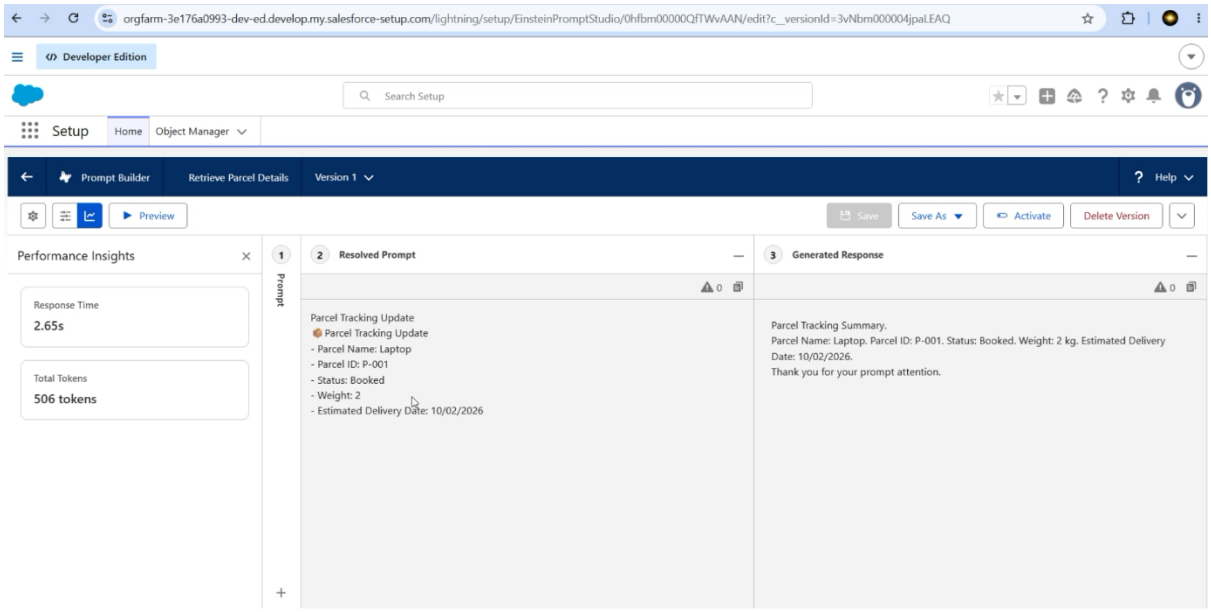
Testing the Prompt

After activating the Prompt Template, it was tested using the Agent preview to verify that the prompt correctly retrieves parcel information.

Steps to Test the Prompt

1. Open the **Agent Preview**.
2. Enter the following query:**“Track parcel P-12345”**
3. The Agentforce AI processes the request and uses the Prompt Template to retrieve the corresponding **Parcel** record.
4. The response should display the parcel details, including:
    - Parcel Name
    - Parcel ID
    - Status
    - Weight
    - Estimated Delivery Date

This confirms that the Prompt Template can retrieve and display parcel information based on the Parcel ID provided by the customer.



Milestone 6 – Flows

A Flow was created to capture the parcel details and return the response from the Prompt Template to the Agentforce AI agent.

Creating the Flow

1. Go to **Setup** in Salesforce.
2. Search for **Flows** in the Quick Find box and open **Flows**.
3. Click **New Flow**.
4. Select **Auto-Launched Flow (No Trigger)** and click **Create**.

Adding Input and Output Variables

5. Create a new resource by clicking **New Resource**.
6. Select **Variable**.
7. Create the following variables:

Variable 1: Ids

- **Data Type:** Text
- Select **Available for Input**.

Variable 2: Output

- **Data Type:** Text
- Select **Available for Output**.

Adding Get Records Element

8. Click the **Add (+)** icon and select **Get Records**.
9. Enter the following details:
    - **Label:** Get Parcel Records
    - **Object:** Parcel
    - **Field:** Parcel_ID__c
    - **Operator:** Equals
    - **Value:** Ids

This retrieves the Parcel record that matches the Parcel ID provided by the Agent.

Adding Action Element

10. Add an **Action** element.
11. Search for and select **Retrieve Parcel Details**.
12. Configure the Action with the following details:

- **Label:** Parcel Updates
- **Object to Summarize:** Parcel from **Get Parcel Records**

The Action uses the Prompt Template to retrieve and format the required parcel information.

Adding Assignment Element

13. Add an **Assignment** element.
14. Enter the following details:

- **Label:** Assignment Outputs
- **Variable:** Output
- **Operator:** Equals
- **Value:** Outputs from Action → Prompt Response

This assigns the response generated by the Prompt Template to the **Output** variable.

Connecting the Flow

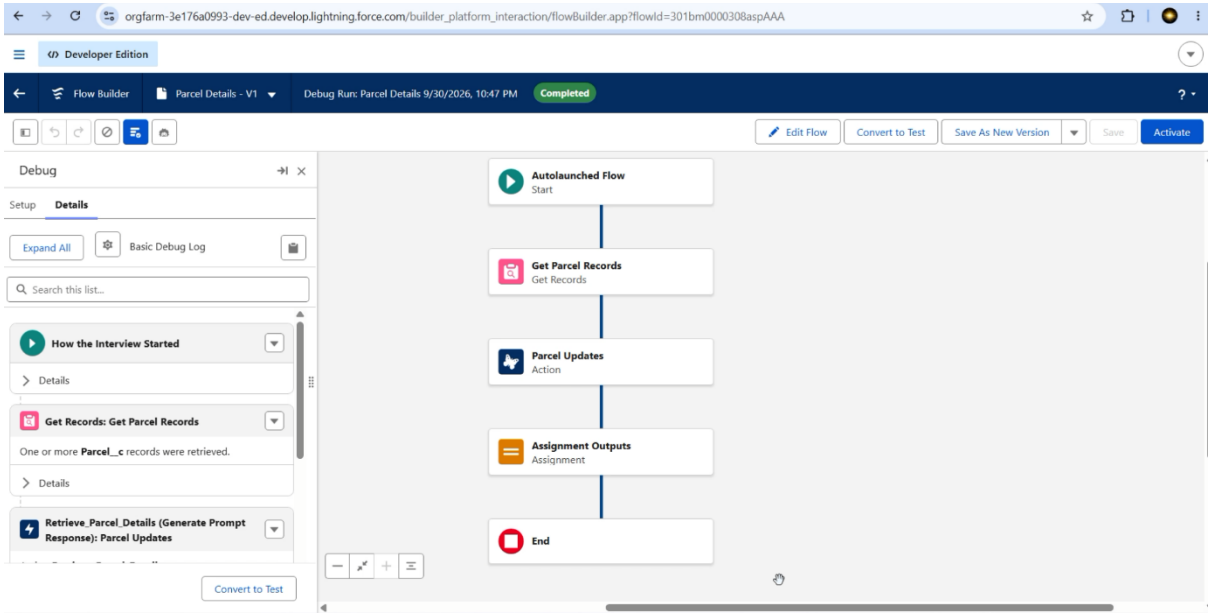
15. Connect the elements in the following order:

Start → Get Records → Action → Assignment → End

Saving and Activating the Flow

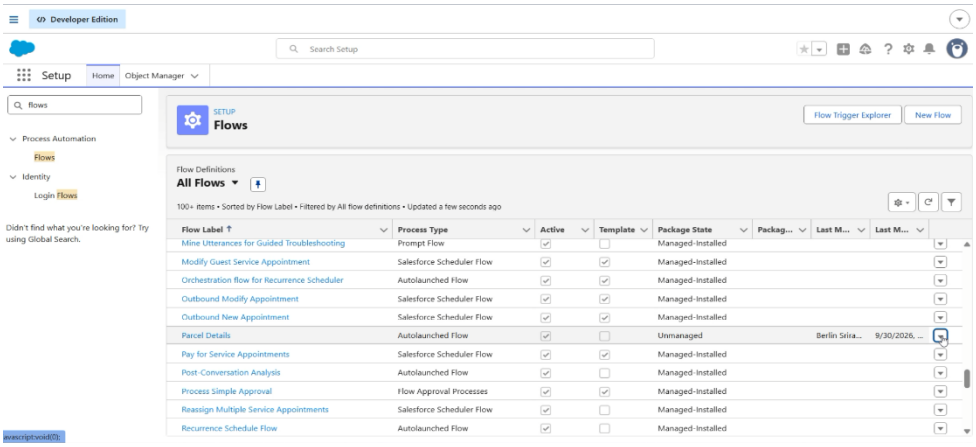
16. Save the Flow with the name **Parcel Details**.

17. Click **Activate** to activate the Flow.



Assigning Flow User Access

18. After saving the Flow, return to the **Flows** page.
19. Click the dropdown next to the newly created **Parcel Details** Flow.
20. Select **Edit Access**.
21. At the top of the page, locate **Flow User Access**.
22. Assign the required available profiles.
23. Click **Save**.



Phase 3: UI/UX Development & Customization

This phase focused on enabling agents to efficiently manage parcel operations while ensuring secure access through **Permission Sets** and AI-driven agent experiences using **Agentforce**.

Milestone 1 – Creating Permission Sets

Permission Sets were configured to provide the required access to the SwiftShip Tracker objects and fields.

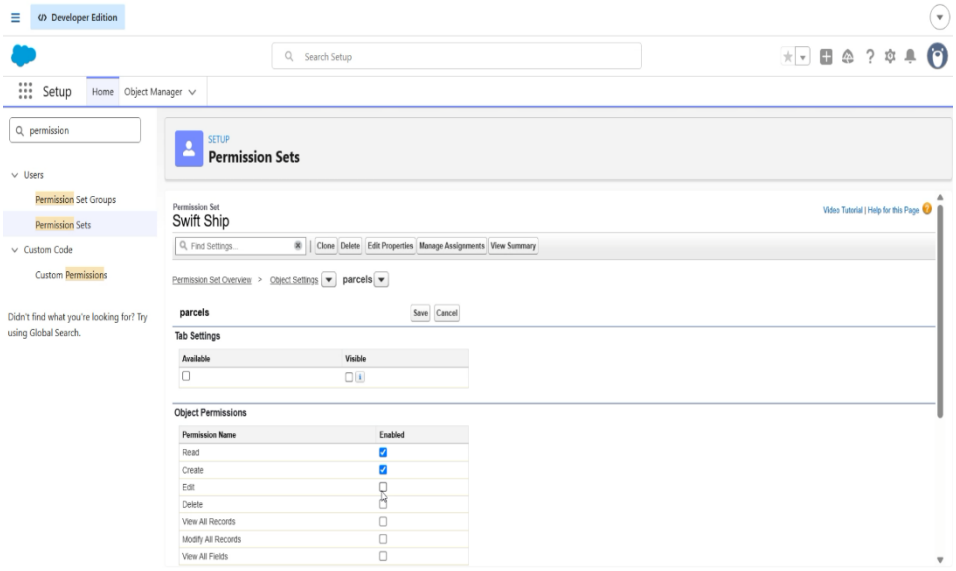
Creating the Permission Set

1. In **Salesforce Setup**, type **Permission Sets** in the **Quick Find** box.
2. Click **Permission Sets** under **Users**.
3. Click **New**.
4. Enter the following details:
    - **Label:** Swift Ship
    - **API Name:** Automatically filled by Salesforce
    - **License:** Einstein Agent
5. Click **Save**.

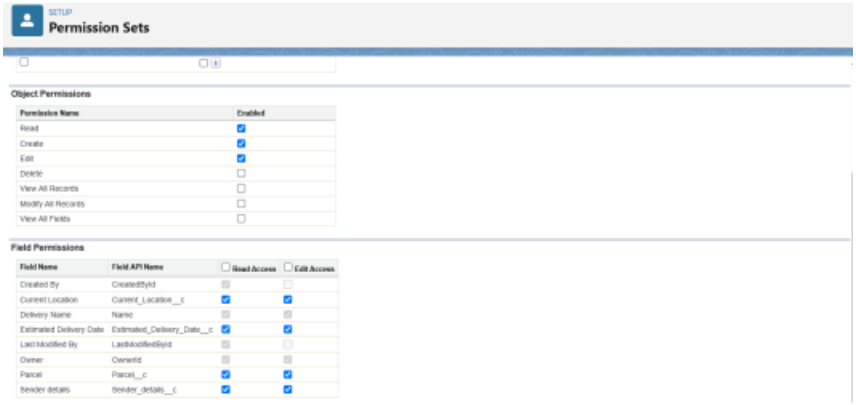
Configuring Object Permissions

1. On the **Swift Ship Permission Set** page, go to **Apps → Object Settings**.
2. Click **Object Settings**.
3. Search for and select each required object one by one.
4. Click **Edit**.
5. Enable the following permissions:
    - **Read**
    - **Create**
    - **Edit**
    - **Delete** *(optional, if agents need permission to remove records)*
6. Under **Field Permissions**, make sure **Visible** is enabled for all required fields.
7. Click **Save** after configuring each object.

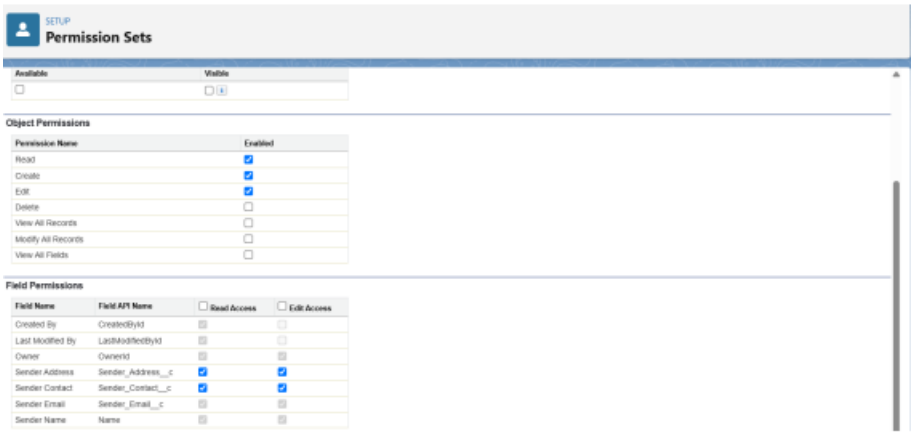
Parcel__c



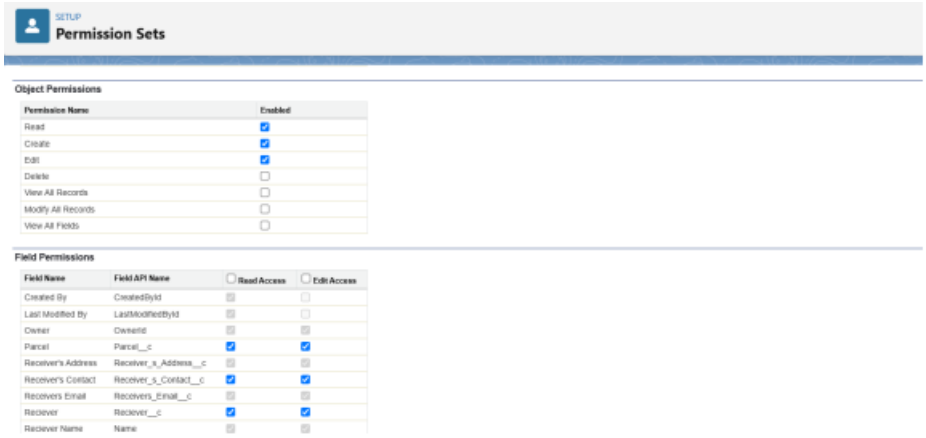
Delivery__c



Sender__c



Receiver__c



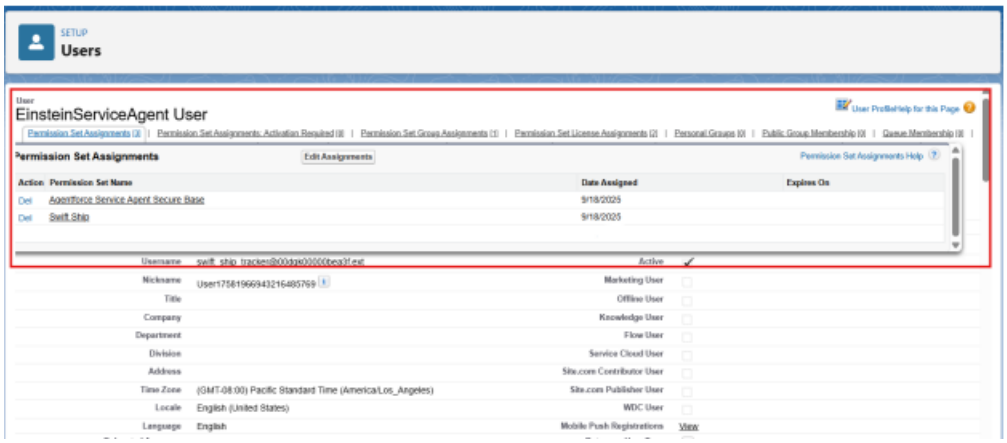
Milestone 2 – Assigning Permissions to the Agent User

The **Swift Ship** Permission Set was assigned to the Agent user to provide the required access to the SwiftShip Tracker objects and features.

Steps to Assign the Permission Set

1. In **Salesforce Setup**, type **Users** in the **Quick Find** box.
2. Click **Users** under the **Users** section.
3. From the list of users, find the **Einstein Service Agent** user representing the Agent, such as **SwiftShip Agent**.
4. Click the **Full Name** of the user to open the user record.
5. Scroll down to the **Permission Set Assignments** section.
6. Click **Edit Assignments**.
7. Under **Available Permission Sets**, find and select **SwiftShip**.
8. Click the **Add** → button to move the permission set to the **Enabled Permission Sets** section.
9. Click **Save**.

The **SwiftShip** Permission Set is now assigned to the Agent user, allowing the user to access the configured Salesforce objects and fields based on the permissions defined in the Permission Set.

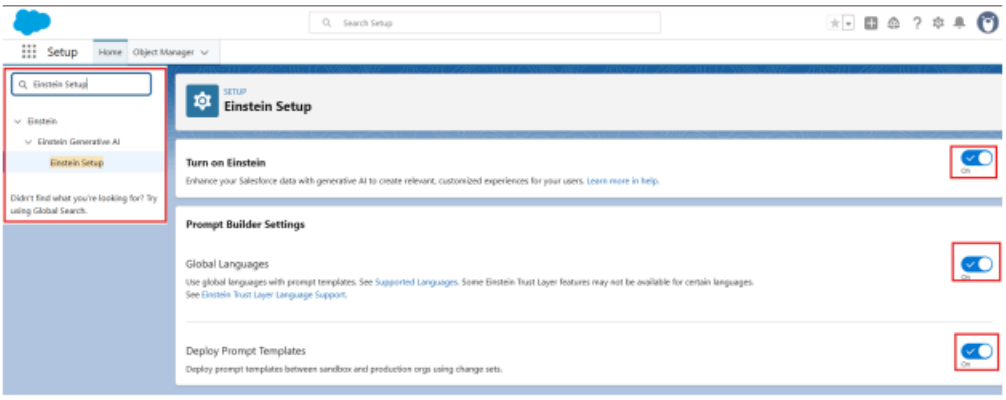


Milestone 3 – Org Setup for Agentforce

Enable Salesforce Default Agent from Salesforce Go

The Salesforce Default Agent was enabled through Salesforce Go to provide the required Agentforce functionality.

Steps:

1. Open Setup.
2. In the Quick Find box, search for Salesforce Go and open it.
3. Search for Default Agent.
4. Click Get Started.
5. Turn on the Default Agent.
6. Refresh the page.
7. 

Enable Einstein Setup

Einstein services were enabled to support Agentforce and Prompt Builder features.

Steps:

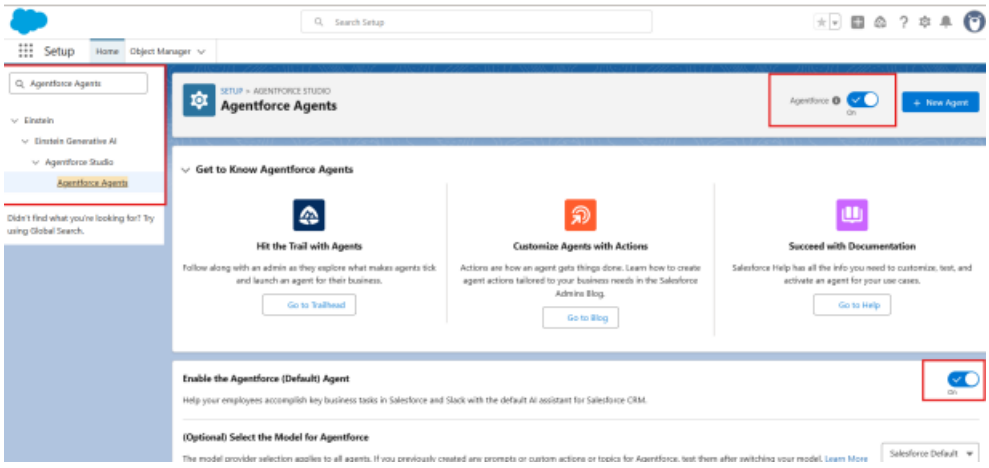
1. Go to Setup → Quick Find and search for Einstein Setup.
2. Open Einstein Setup.
3. Click Get Started if prompted.
4. Turn on the following services:
    - Turn On Einstein
    - Global Languages
    - Deploy Prompt Template
5. Refresh the browser page.

Enable Agentforce

After enabling Einstein services, Agentforce was enabled from the Agentforce Agents settings.

Steps:

1. Go to Setup → Quick Find.
2. Search for Agentforce Agents and open it.
3. Turn on the following options:
    - Agentforce
    - Enable Agentforce Default Agent
4. Refresh the page to apply the changes.



Milestone 4 – Building the Agent

Opening the Agent Builder

The Agentforce Default Agent was opened in **Agent Builder** to configure subagents and actions.

Steps:

1. Go to **Setup**.
2. In the **Quick Find** box, search for **Agentforce Agents**.
3. Click **Agentforce Agents**.
4. Select the **Default Agent**.
5. Click **Open in Builder**.

Assign Subagents to the Agent

Subagents were created and assigned to the Agent to handle specific tasks.

Steps:

1. In **Agent Builder**, go to the **Subagents** section.
2. Click **New**.
3. Select **New Subagent**.
4. Create the required subagent.
5. Enter the subagent details as shown in the configuration.

Adding Actions to Agentforce Assets

Actions were added to the Agentforce assets so that the agent can perform the required operations using Salesforce Flows.

Steps:

1. In **Agent Builder**, open the required **Topic**.
2. Go to **Topic Actions**.
3. Click **New Agent Action**.
4. For **Action Type**, select **Flow**.
5. Click **Next**.
6. Under **Reference Flow**, select the recently created Flow.
7. Click **Next**.
8. Enter the required details under **Agent Configuration**.

9. Save the Agent Action.

Assign Subagent Action

The created Flow Action was added to the **Parcel Updates** subagent through the Agentforce Asset Library.

Steps:

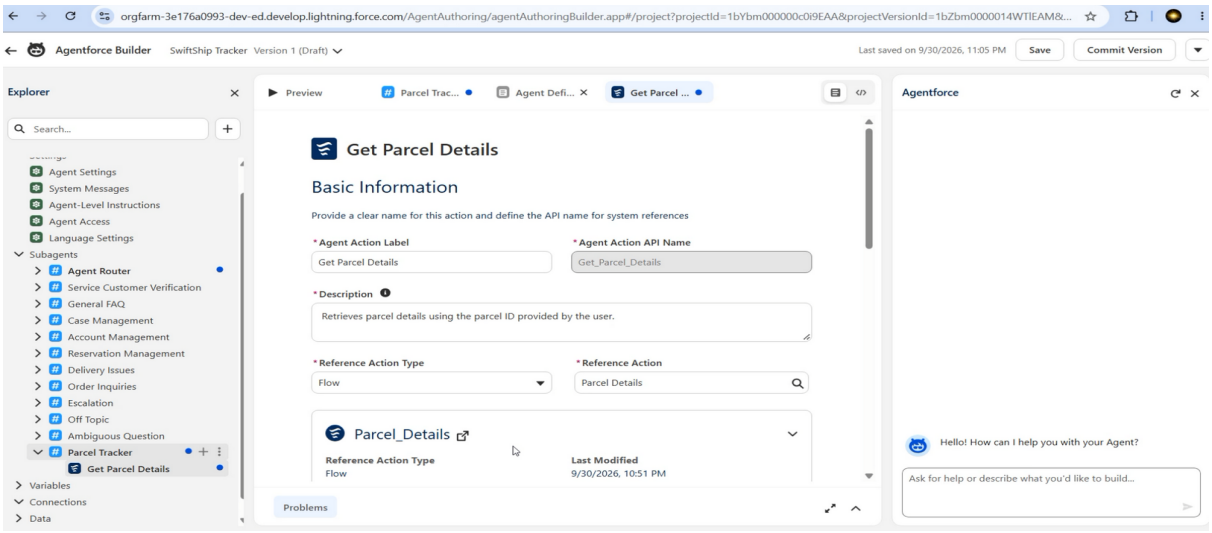
1. In **Agent Builder**, go to the **Subagents** section.
2. Click the **Parcel Updates** subagent.
3. Open **Subagent Actions**.
4. Click **Add from Asset Library**.
5. Select the previously created **Flow Action**.
6. Click **Finish**.

Rename and Activate the Agent

The Agent was renamed and activated after completing the required configuration.

Steps:

1. In **Agent Builder**, click **Settings**.
2. Rename the Agent to **SwiftShip Tracker**.
3. Click **Save**.
4. Click **Ignore and Activate**.
5. Click **Activate** to activate the Agent.



Phase 4: Data Migration, Testing & Security

This phase focused on validating the functionality and reliability of the **Parcel Tracking Agent** created in Agent Builder. The testing ensured that the agent correctly identifies topics, triggers the assigned Flow (**SwiftShip Tracker**), retrieves or updates parcel information, and performs the required actions without access or permission issues.

The main objective was to confirm that the AI-driven agent works correctly within Salesforce and provides accurate, real-time responses to parcel management queries.

Verify Agent

The Agent was verified to ensure that it was created and available in Salesforce.

Steps:

1. Go to **Setup → Agentforce Agents**.
2. Open the **Agents** list.
3. Confirm that the **Parcel Tracking Agent** appears in the list.

Validate Assigned Topic

The assigned Subagent and its related topic were verified in Agent Builder.

Steps:

1. Open **Agent Builder**.
2. Go to the **Subagent** section.
3. Confirm that the created Subagent, such as **Parcel Tracking** or **SwiftShip Tracker**, is listed.
4. Open the topic and verify that:
    - It contains a prompt or description related to parcel management.
    - The **Action/Flow (SwiftShip Tracker)** is linked to the topic.
5. Click **Save**.
6. Ensure that the topic is **Active**.

Check the Flow Association

The Flow associated with the Agent was checked to ensure that it was activated and configured correctly.

Steps:

1. Go to **Setup → Flows**.
2. Locate the Flow named **SwiftShip Tracker**.

3. Confirm that the Flow is **Activated**.
4. Verify that the Flow includes the required screens or actions for:
    - Parcel ID input
    - Parcel details retrieval
    - Status update or confirmation message
5. Open the Flow's **Advanced Settings**.
6. If required for data access, ensure that **Run in System Context (Without Sharing)** is enabled.

Activate and Test the Agent

The Agent was activated and tested using sample parcel-related queries.

Steps:

1. Go to **Setup → Agent Builder → Parcel Tracking Agent**.
2. Click **Activate** if the Agent is not already active.
3. Once activated, open **Preview / Try It Out** mode.
4. Enter a test query such as:
    - Track parcel ID P12345
    - Update delivery status for parcel P67890 to Delivered.
5. Observe the Agent's response.
6. Verify that the Agent correctly identifies the request and triggers the associated Flow.

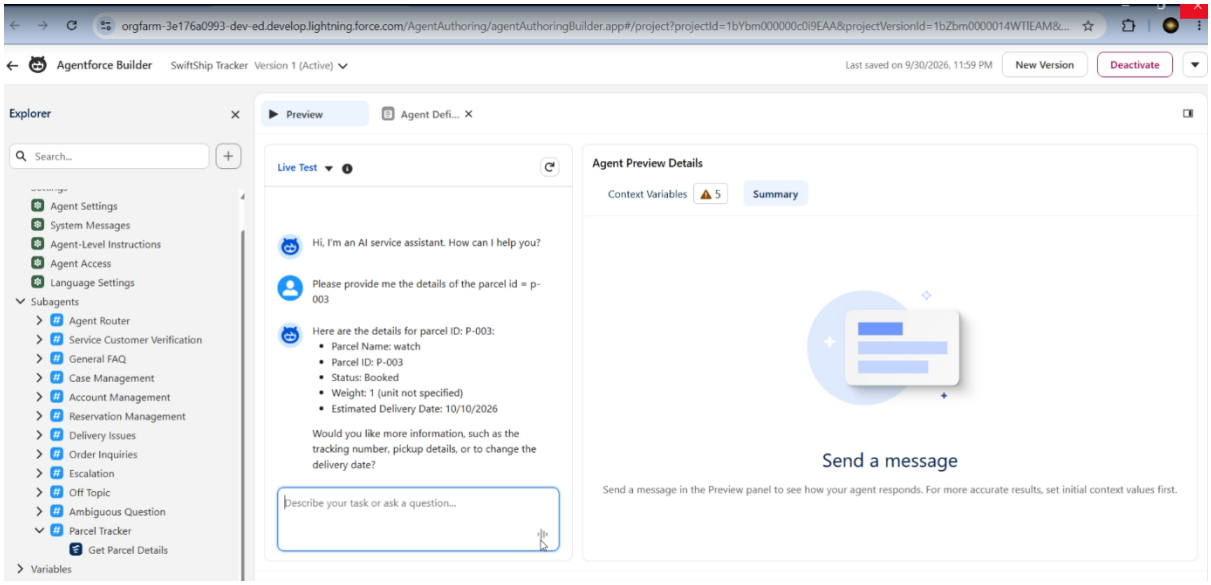
Validate Data in Salesforce

After the Agent executed the Flow, the corresponding parcel data was checked in Salesforce to confirm that the expected changes were made.

Steps:

1. Go to **App Launcher → Parcels (Parcel__c)**.
2. Search for the **Parcel ID** used during testing.
3. Open the corresponding Parcel record.
4. Verify that:
    - The **Status** or related field has been updated correctly.
    - Any records created by the Flow, such as bookings or updates, appear correctly.

5. Confirm that the data displayed in Salesforce matches the response provided by the Agent.



Phase 5: Deployment, Documentation & Maintenance

Deployment and Validation

As part of the **SwiftShip Tracker (Agentforce)** project, this phase focused on validating the **Parcel Tracking Agent** created using Salesforce Agent Builder within the Salesforce Developer Org environment.

The main objective was to verify that the AI-powered Service Agent works as intended, including:

- Correct recognition of topics and user queries.
- Proper execution of the assigned Salesforce Flows.
- Correct retrieval of data from Salesforce custom objects.
- Proper functioning of permissions and Agent interactions.

Testing was performed within the **Salesforce Developer Org** to simulate the Agent's behaviour in a live environment. This helped verify that the configured permissions, Flows, topics, actions, and Salesforce interactions were working correctly before deployment.

Deployment Scope

Real-world production testing, such as **User Acceptance Testing (UAT)** or sandbox deployment, was not performed as part of this project because:

- Salesforce Developer Edition orgs are standalone environments and do not provide production-linked testing environments.
- Enterprise-level Agentforce deployments generally involve sandbox testing, staging environments, and CI/CD automation, which are outside the scope of this academic/prototype implementation.

Maintenance, Monitoring & Troubleshooting

In the **SwiftShip Tracker (Agentforce)** project, ongoing maintenance is important to ensure that **Parcel, Customer, and Delivery** data remains accurate, synchronised, and up to date.

Regular monitoring helps identify issues such as:

- Incorrect or misrouted Agent queries.
- Failed Flow executions.
- Delays in Agentforce responses.
- Configuration-related issues.
- Incorrect or outdated data.

Troubleshooting activities include:

- Resolving configuration errors.
- Updating Flow logic when required.
- Refining Agent topics.
- Improving prompts for better user interactions.
- Checking Agent and Flow behaviour after configuration changes.

Although this project was implemented in a **Salesforce Developer Org for learning purposes**, these maintenance and monitoring practices are important in real-world enterprise deployments. They help ensure that AI-powered service agents remain reliable, scalable, and aligned with changing customer service operations and business requirements.

7. FUNCTIONAL & PERFORMANCE TESTING

During the testing phase, screenshots were captured from Salesforce to verify the functionality and performance of the **SwiftShip Tracker** system.

The following features were tested:

- Parcel creation and management.
- Sender and receiver information.
- Parcel and delivery data validation.
- Parcel status updates.
- Salesforce Flow execution and automation.
- Agentforce Agent and Prompt Builder functionality.
- Scheduled and automated processes.
- Reports and dashboard analytics.

All major features were tested to ensure that they performed correctly and produced the expected results.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Centralises parcel, customer, sender, receiver, and delivery information in Salesforce.
- Provides better visibility into parcel booking and delivery tracking.
- Automates important parcel and delivery processes using Salesforce Flow.
- Agentforce enables conversational customer support and parcel-status assistance.
- Prompt Builder supports AI-powered responses and self-service tracking.
- Reduces manual effort for customers, support teams, and administrators.
- Provides timely notifications and updates throughout the delivery process.
- Experience Cloud can provide customers with convenient access to parcel tracking information.
- Supports integration with external logistics systems.
- Can be scaled to support multiple locations and growing parcel operations.

Disadvantages

- Requires accurate and up-to-date parcel and customer data.
- Depends on Salesforce configuration and automation.
- Agentforce responses depend on properly configured prompts and available data.
- External logistics integrations may increase implementation complexity.

- Flow and automation logic require regular maintenance when business processes change.
- Users need appropriate Salesforce or Experience Cloud access to use the system.
- Initial Salesforce implementation and configuration can require significant effort.
- Expanding the system to additional locations or integrations may require further customisation.

9. CONCLUSION

The **SwiftShip Tracker** project demonstrates how Salesforce can be used to build a centralised parcel management and tracking solution. The project integrates **custom objects, relationships, Salesforce Flows, Apex, Email Alerts, Experience Cloud, Agentforce AI, Prompt Builder, security controls, reports, and dashboards** to support parcel booking, delivery tracking, status updates, and customer communication.

The integration of **Agentforce AI, Prompt Builder, and Salesforce Flow** provides a conversational approach to parcel tracking. Customers can obtain shipment information through the Agent without depending entirely on manual support.

Overall, the project provides a practical foundation for a scalable and user-friendly parcel management solution.

10. FUTURE SCOPE

The SwiftShip Tracker system can be further enhanced in the following areas:

- Extend **Agentforce** capabilities to handle more parcel-management queries.
- Enhance real-time delivery tracking and location visibility.
- Expand customer self-service capabilities through **Experience Cloud**.
- Add advanced analytics for delivery performance and operational insights.
- Integrate additional external logistics and delivery systems.
- Improve AI prompts and conversational experiences for more effective customer interactions.
- Introduce enterprise-level sandbox and CI/CD deployment.
- Expand automation for additional parcel operations.

The source identifies enterprise deployment, continuous monitoring, prompt refinement, Flow improvement, and AI-agent scalability as areas for future enhancement.

1. APPENDIX

A. Custom Objects

- **Parcel**
- **Delivery**
- **Sender**
- **Receiver**

B. Standard Objects

- **Account**
- **Contact**
- **Case**
- **Task**
- **EmailMessage**
- **User**

C. Automation

- **Salesforce Flows**
- **Approval Processes**
- **Email Alerts**

D. Apex

- **Apex Classes**
- **Batch Apex**

E. AI

- **Agentforce AI**
- **Prompt Builder**

F. User Interface

- **Lightning App**
- **Lightning Record Pages**
- **Experience Cloud**

G. Security

- **Profiles**
- **Roles**
- **Permission Sets**
- **Sharing Rules**
- **Field-Level Security**

H. Analytics

- **Reports**
- **Dashboards**

I. Testing

- **Flow Testing**
- **Agentforce Testing**
- **Data Retrieval Validation**
- **Security and Permission Validation**

Thank You